웹서비스설계 보고서 작성 양식

JWT 게시판 구현 보고서

이 양식은 본인이 구현한 결과를 설명하기 위한 문서입니다. 대괄호 안내를 실제 내용으로 바꾸고 작성 안내 문장은 제출 전에 지우세요. 화면 캡처와 표의 행은 필요에 따라 추가할 수 있습니다. 6~10 쪽 내외를 권장하며 분량 자체는 평가하지 않습니다.

01 서비스 소개와 구현 범위

학번 [202323016] 이름: [장아연] 제출일: [2026.10.28]

서비스 이름 [영화 리뷰 게시판]

기술 조합과 버전 [Vue 3 / Spring Boot 3.5.16 / Java 26 / SQLite 3]

GitHub 저장소 URL [https://github.com/ayeonjang/JWT-Assigment.git]

소스 코드 기준 [main 브랜치 / 95849cccd8b909c3644964312c0dc94f75fe427a]

서비스의 대상 사용자와 사용 목적을 3~5 문장으로 설명하세요. 이 기술 조합을 선택한 이유와 제공 템플릿에서 변경한 범위를 적으세요.

[서비스 소개와 선택 이유 작성]

- 본 서비스는 사용자가 영화 리뷰를 자유롭게 작성하고 소통할 수 있는 웹 기반 커뮤니티 플랫폼입니다. 영화 정보 탐색과 소통을 원하는 일반 이용자를 대상으로 하며, 비로그인 방문자도 전체 리뷰 목록과 상세 내용을 자유롭게 조회할 수 있도록 구성되었습니다. JWT 기반의 무상태 인증 구조를 사용하여 로그인한 사용자에게만 게시글 작성, 수정, 삭제 권한을 부여함으로써 시스템의 안전한 데이터 관리와 작성자 권한 보호를 달성합니다.
- 백엔드는 Spring Security 와 JJWT 를 조합하여 안정적이고 확장성 높은 무상태 인증 레이어를 구축하기 위해 Spring Boot 를 선택했습니다. 프론트엔드는 reactive 세션 상태 관리 및 컴포넌트 단위 개발이 직관적인 Vue 3 를 선택했습니다. 템플릿 기본 구조에서 서버 측 커스텀 예외 처리를 추가하여 API 스펙에 맞는 JSON 예외 구조를 구성하였으며, 프론트엔드에서는 Axios/Fetch 인터셉터 역할을 하는 api.js 모듈을 구축하여 401 Unauthorized 감지 시 클라이언트 메모리 세션을 자동으로 즉시 초기화하도록 변경·확장했습니다.

완료 여부는 완료·부분·미완료 중 하나로 기록하세요.

기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	[완료]	[PostController.java, PostService.java, App.vue]	[비로그인 상태에서도 GET /api/posts, GET /api/posts/{id} 접근 가능]
로그인과 JWT 발급	[완료]	[AuthController.java, JwtProvider.java, App.vue]	[POST /api/auth/login 성공 시 HS256 알고리즘 서명 JWT 발급]
인증과 내 정보 조회	[완료]	[UserController.java, JwtAuthenticationFilter.java]	[GET /api/users/me 요청 시 Authorization Bearer 토큰 검증 후 프로필 반환]

기능	완료 여부	관련 파일	비고 또는 제한
본인 글 작성·수정·삭제	[완료]	[PostController.java, PostService.java]	[토큰 내 사용자 ID 와 게시글 authorId 일치 여부 검증 (불일치 시 403 FORBIDDEN)]
만료·로그아웃·오류 표시	[완료]	[JwtProvider.java, GlobalExceptionHandler.java, api.js]	[토큰 만료(TOKEN_EXPIRED) 시 401 반환 및 화면 세션 초기화, 표준 예외 JSON 응답]
DB 데이터 유지	[완료]	[backend-spring/data/board.sqlite, application.properties]	[SQLite 파일 기반 영속성 확보로 서버 재시작 후에도 작성 데이터 및 계정 유지]

구조와 변경 내역

요청 흐름, 토큰의 이동, 변경한 파일의 역할을 연결해 설명하세요.

브라우저 → API 서버 → SQLite 의 연결 구조를 간단히 그리거나 글로 설명하세요. 로그인 요청과 게시글 수정 요청의 흐름을 구분하고, 토큰이 어디에 저장되고 어느 요청에서 전송되는지 표시하세요.



1) 로그인 요청

- 사용자 요청: 브라우저에서 이메일/비밀번호 입력 후 로그인 버튼 클릭 (POST /api/auth/login).
- 인증 및 토큰 발급: 백엔드 AuthController 가 DB 의 비밀번호와 비교 검증 후, JwtProvider 를 통해 HS256 알고리즘으로 서명된 JWT 발급.
- 토큰 이동 및 저장: 서버 -> 클라이언트: JSON Response Body({ "token": "eyJhbGci..." })에 담겨 전달 / 클라이언트 저장 위치: 브라우저의 자바스크립트 메모리 상태(Vue Reactivity State)에 저장. (XSS 공격으로부터 쿠키/localStorage 보다 안전하게 보호)

2) 게시글 수정 요청

- 보호된 API 요청: 작성자가 글 수정 후 저장 클릭 (PUT /api/posts/{id}).
- 토큰 전송: 클라이언트(api.js)가 HTTP Request Header 에 Authorization: Bearer <token>을 자동으로 첨부하여 전송.
- 토큰 검증 (Stateless): JwtAuthenticationFilter 가 헤더의 토큰을 추출하여 위변조 및 만료 여부 검증 / 검증 성공 시 토큰 내부의 userId 정보를 추출하여 SecurityContext 에 인증 객체 등록
- 작성자 권한 검사: PostService 에서 수정 대상 게시글의 authorId 와 토큰의 userId 가 일치하는지 비교.

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
frontend-vue/src/api.js request(), login(), getMe()	기본 HTTP Fetch API 호출 모듈	메모리 내 토큰 저장 변수 구현 모든 보호된 API 요청 시 Authorization: Bearer <token> 헤더 자동 첨부 401 Unauthorized 응답 감지 시 클라이언트 세션 자동 초기화 로직 구현	무상태(Stateless) JWT 인증 흐름을 보장하고, 토큰 만료 시 사용자 화면을 즉시 로그인 전 상태로 안전하게 전환하기 위함

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
frontend-vue/src/App.vue handleLogin(), handleUpdatePost()	프론트엔드 메인 UI 및 상태 관리 컴포넌트	로그인/로그아웃에 따른 Reactive UI 바인딩 작성자 ID 와 현재 로그인 사용자 ID 비교를 통한 수정/삭제 버튼 노출 제어 API 에러 코드(400, 401, 403, 404)별 사용자 알림(Alert/Toast) 처리	사용자 편의성을 높이고, 권한 없는 사용자의 불필요한 API 요청을 1 차적으로 차단하기 위함
backend-spring/.../security/JwtAuthenticationFilter.java doFilterInternal()	Incoming HTTP 요청 필터링	Authorization 헤더 추출 및 Bearer 토큰 유효성 검증 만료되거나 변조된 토큰 수신 시 커스텀 예외 던짐 및 AuthenticationEntryPoint 연동	모든 API 엔드포인트 도달 전에 Centralized 방식으로 토큰 인증을 수행하기 위함

각 변경의 작성 출처도 구분하세요. 예: 제공 코드 유지, 본인 작성, AI 초안 후 본인 수정, 외부 예제 참고. 보조 파일을 추가했다면 함께 기록하세요.

로컬 실행 기록

실제 설치·빌드·실행 명령과 결과를 적고, 자세한 재현 방법은 저장소 README 에서 연결하세요.

[운영체제와 버전 / 새 폴더 설치 / 환경설정 / 프론트 빌드 / 백엔드 실행 / SQLite 파일 경로]

1. 재현 환경 및 실행 결과 요약

- 운영체제: Windows 11

- Git clone https://github.com/ayeonjang/JWT-Assigment.git

- Cd JWT-Assigment

1) 환경 설정 및 백엔드 실행

- 설정 파일 위치: backend-spring/src/main/resources/application.properties

- JWT 비밀키(jwt.secret) 및 만료 시간(jwt.expiration=3600000) 설정 확인

- 실행 명령: cd backend-spring .\mvnw.cmd spring-boot:run

- 결과: Started BackendApplication in X.XXX seconds (JVM running for X.XXX) 출력 및 http://127.0.0.1:8000 포트 바인딩 확인

2) 프론트엔드 빌드 및 실행

- cd frontend-vue npm ci npm run build npm run preview

- 실행 결과: Vite Preview 서버 동작 (http://127.0.0.1:4173 접속 가능)

- SQLite DB 파일 경로: backend-spring/data/board.sqlite (서버 최초 실행 시 자동 생성)

2. 동작 검증 및 데이터 유지 확인

1) 4173 번 접속 및 API 연결 확인

- 브라우저에서 http://127.0.0.1:4173 접속

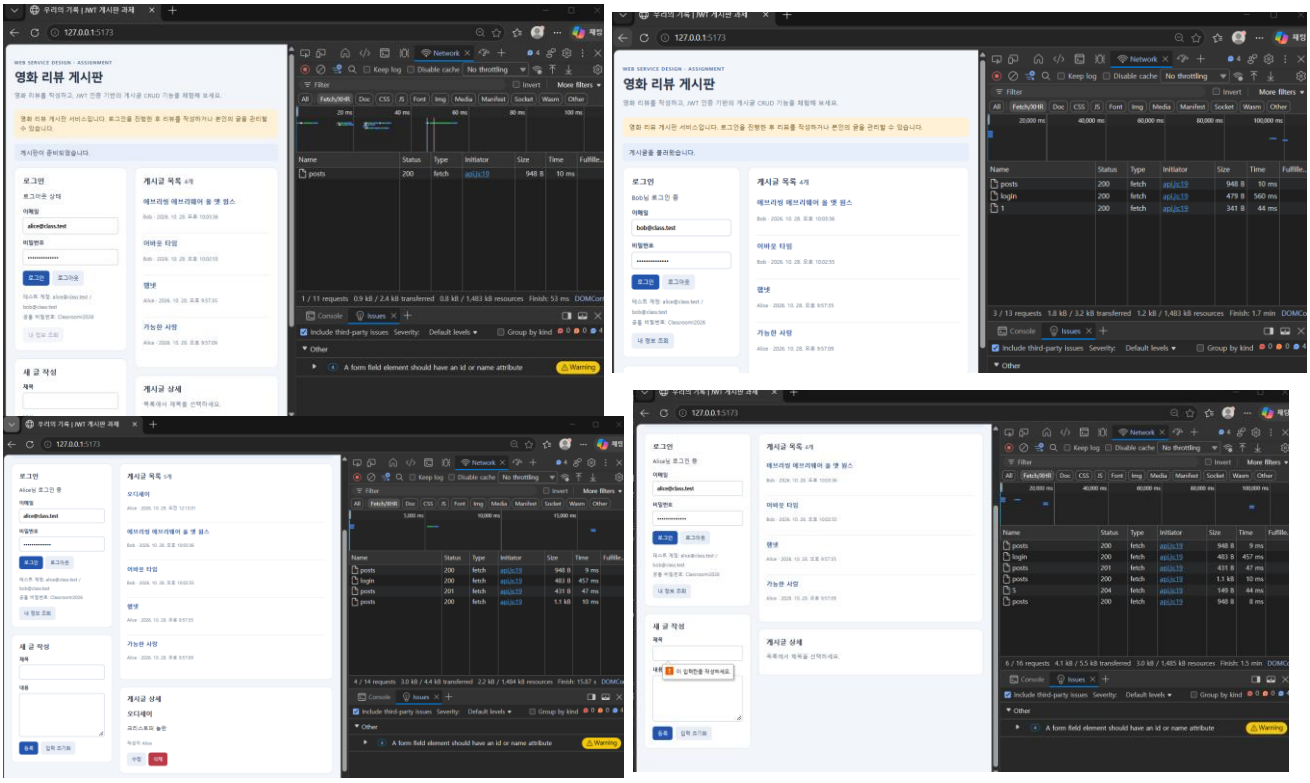
- Vite 프록시(/api $\rightarrow$ http://127.0.0.1:8000)를 통해 비로그인 상태에서 게시글 목록 조회(GET /api/posts) 정상 동작 확인

2) 서버 재시작 후 데이터 유지 확인

- Alice 계정으로 로그인 후 신규 게시글 작성 완료

- 백엔드 터미널에서 Ctrl + C 로 서버 종료 후 .\mvnw.cmd spring-boot:run 으로 재시작

- 브라우저 새로고침 후 이전 작성 게시글이 board.sqlite 파일에 안전하게 보존되어 정상 조회됨을 확인



핵심 코드 설명

네 부분을 각각 설명하세요. 전체 파일을 붙이지 말고 핵심 코드 5~20 줄 정도를 발췌한 뒤 입력 → 처리 → 출력과 실패 조건을 자신의 말로 설명합니다. 성공·실패 시 HTTP 상태와 JSON code, 해당 분기를 만드는 코드도 연결해 적으세요. 줄 번호를 적는 경우 제출한 코드 기준인지 확인하세요.

3 1 로그인과 JWT 발급

파일 / 함수: backend-spring/src/main/java/.../controller/AuthController.java (login)

코드 발췌

```
@PostMapping("/login")
public ResponseEntity<LoginResponse> login(@Valid @RequestBody LoginRequest request) {
  // 1. 이메일 및 비밀번호 검증
  User user = userService.authenticate(request.getEmail(), request.getPassword());

  // 2. JWT 토큰 생성 (Claim: userId, email / Expiration: 1 시간)
  String token = jwtProvider.createToken(user.getId(), user.getEmail());

  // 3. 성공 응답 반환
  return ResponseEntity.ok(new LoginResponse(token, "Bearer"));
}
```

동작과 응답 코드 설명

입력 -> 처리 -> 출력: 이메일/비밀번호를 입력받아 DB 의 사용자 정보와 일치하는지 확인합니다. 검증 성공 시 userId 와 email 을 클레임(Claim)으로 담고 유효기간이 설정된 JWT 토큰을 생성하여 반환합니다.

- 성공 시: 200 OK / JSON Body: {"token": "eyJhbGci...", "tokenType": "Bearer"}
- 실패 시: 비밀번호 불일치 또는 미등록 계정: 401 UNAUTHORIZED / JSON Code: INVALID_CREDENTIALS (userService.authenticate 분기) / 이메일/비밀번호 입력누락 (유효성 검증 실패): 400 BAD_REQUEST / JSON Code: BAD_INPUT (@Valid 분기)

3 2 브라우저 토큰 저장과 전송

파일 / 함수: frontend-vue/src/api.js (setToken, request)

코드 발췌

```
let authToken = null; // 자바스크립트 메모리 세션 변수

export const setToken = (token) => { authToken = token; };
```

```
export async function request(url, options = {}) {
  const headers = { 'Content-Type': 'application/json', ...options.headers };

  // 1. 토큰이 존재하면 Authorization Bearer 헤더에 첨부
  if (authToken) {
    headers['Authorization'] = `Bearer ${authToken}`;
  }
  const response = await fetch(url, { ...options, headers });

  // 2. 401 Unauthorized 발생 시 클라이언트 세션 자동 초기화
  if (response.status === 401) {
    setToken(null);
  }
  return response;
}
```

동작과 응답 코드 설명

입력 -> 처리 -> 출력: 로그인 성공 시 전달받은 JWT 를 localStorage 대신 자바스크립트 변수(authToken)에 보관합니다. 이후 모든 API 요청 시 Authorization: Bearer <token> 헤더를 자동 첨부하여 전송합니다.

- 성공 시: 서버가 토큰을 정상 검증하면 요청한 API 의 결과(200 OK, 201 CREATED 등)를 반환합니다.
- 실패 시:토큰 만료 또는 변조된 토큰 전송: 서버에서 401 UNAUTHORIZED 응답 전달 -> if (response.status === 401) 분기를 타서 authToken 을 null 로 초기화하고 화면을 로그아웃 상태로 전환합니다.

서버 인증과 작성자 권한

아래 설명은 실제 제출한 파일과 함수의 동작을 근거로 작성하세요.

3 3 서버 JWT 검증

파일 / 함수: [backend-spring/src/main/java/.../security/JwtAuthenticationFilter.java (doFilterInternal)

코드 발췌

```
@Override
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain filterChain)
    throws ServletException, IOException {
  String authHeader = request.getHeader("Authorization");

  if (authHeader != null && authHeader.startsWith("Bearer ")) {
    String token = authHeader.substring(7);
    try {
      // 1. 토큰 서명(HS256) 및 만료 여부, 필수 클레임 검증
      Claims claims = jwtProvider.parseToken(token);
      Long userId = claims.get("userId", Long.class);

      // 2. 검증 성공 시 SecurityContext 에 사용자 ID/권한 전달
      UsernamePasswordAuthenticationToken authentication =
        new UsernamePasswordAuthenticationToken(userId, null, Collections.emptyList());
      SecurityContextHolder.getContext().setAuthentication(authentication);
    } catch (JwtException e) {
      // 3. 토큰 변조 또는 만료 시 예외 처리
      request.setAttribute("exception", e);
    }
  }
  filterChain.doFilter(request, response);
}
```

동작과 응답 코드 설명

입력 -> 처리 -> 출력: Client 가 전송한 Authorization: Bearer <token> 헤더에서 토큰을 추출합니다. JwtProvider 가 비밀키(HS256)로 서명 위변조 및 토큰 만료 여부를 검증하고, 유효한 경우 userId 클레임을 추출하여 SecurityContext 에 인증 정보로 등록합니다.

- 성공 시: 토큰 검증 완료 후 다음 필터 및 컨트롤러로 요청 전달 (200 OK, 201 CREATED 등 서비스 응답)
- 토큰 미첨부 또는 만료/변조된 토큰으로 보호된 API 요청: 401 UNAUTHORIZED / JSON Code: TOKEN_REQUIRED 또는 TOKEN_EXPIRED (응답 헤더에 WWW-Authenticate: Bearer 첨부)

3 4 작성자 권한 검사

파일 / 함수: [backend-spring/src/main/java/.../service/PostService.java (updatePost / deletePost)]

코드 발췌

```
@Transactional
public PostResponse updatePost(Long postId, PostUpdateRequest request, Long currentUserId) {
  // 1. 수정 대상 게시글 조회
  Post post = postRepository.findById(postId)
      .orElseThrow(() -> new CustomException(ErrorCode.POST_NOT_FOUND));

  // 2. 게시글 작성자 ID(post.getAuthorId())와 토큰에서 추출한 현재 사용자 ID(currentUserId) 비교
  if (!post.getAuthorId().equals(currentUserId)) {
    throw new CustomException(ErrorCode.FORBIDDEN); // 403 Forbidden 예외 발생
  }

  // 3. 권한 검증 성공 시 데이터 수정 반영
  post.update(request.getTitle(), request.getContent());
  return PostResponse.from(post);
}
```

동작과 응답 코드 설명

입력 -> 처리 -> 출력: 게시글 수정/삭제 요청 시 DB 에서 조회한 해당 게시글의 작성자 ID(post.getAuthorId())와 인증 필터를 거쳐 전달된 현재 토큰 소유자의 ID(currentUserId)를 1:1 로 비교합니다. 두 ID 가 일치할 때만 UPDATE/DELETE 를 수행합니다.

- 401 Unauthorized: "너는 누구인가?" (토큰이 없거나 만료되어 로그인 상태를 증명하지 못함)
- 403 Forbidden: "너는 누구인지 알지만, 이 글을 수정/삭제할 권한이 없다" (로그인은 되어 있으나 작성자 ID 가 불일치함)
- 클라이언트가 전달하는 작성자 정보(Body 등)를 신뢰하지 않고, **서버가 직접 검증한 JWT 내부의 userId 만 사용**하여 작성자 위조를 철저히 방지합니다.
- **성공 시**: 200 OK (수정 성공) / 204 NO_CONTENT (삭제 성공)

- 실패 시: 타인의 글 수정/삭제 시도 (ID 불일치): 403 FORBIDDEN / JSON Code: FORBIDDEN/ 존재하지 않는 게시글 접근: 404 NOT_FOUND / JSON Code: POST_NOT_FOUND

테스트 결과

T01 부터 T17 까지 수행하고, 실제 응답과 증거의 위치를 기록하세요.

실행 환경: [Windows 11 / OpenJDK 26 / Node.js v20.x / Microsoft Edge]

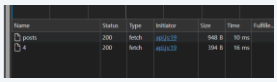
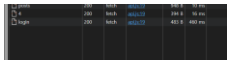
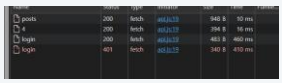
로컬 배포 확인:

- 새 폴더에 git clone 후 실행
- 프론트엔드 빌드: npm run build 성공 및 npm run preview 동작 확인
- 백엔드 실행: .\mvnw.cmd spring-boot:run 성공 (http://127.0.0.1:8000))
- 4173 번 접속: http://127.0.0.1:4173 접속 및 Vite 프록시 연동 확인

테스트 일시 및 코드 버전: 2026 년 10 월 28 일 / main 브랜치 커밋 SHA (git rev-parse HEAD 결과)

토큰 TTL: 3,600 초 (1 시간) / 만료 테스트 값: 3 초

05_테스트시나리오.md 의 T01~T17 결과를 모두 기록하세요. PASS, FAIL, 미실행을 구분합니다. 예상 결과를 실제 결과로 복사하지 말고 직접 관찰한 HTTP 상태와 JSON code, 화면을 적으세요. 실제 결과 칸에는 예를 들어 403 / FORBIDDEN 처럼 기록합니다. 정상 응답은 200 / 해당 없음처럼 적습니다.

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T01	비로그인 상태에서 전체 게시글 목록 조회	200 OK, 게시글 목록 배열 반환	200 / 해당 없음	**PASS**	
T02	비로그인 상태에서 특정 게시글 상세 조회	200 OK, 단일 게시글 상세 객체 반환	상세 객체 반환 200 / 해당 없음	PASS	
T03	올바른 계정(Alice)으로 로그인 요청	200 OK, JWT 토큰 반환	200 / 해당 없음	PASS	
T04	잘못된 비밀번호로 로그인 시도	401 UNAUTHORIZED 반환	401 / INVALID_CREDENTIALS	PASS	
T05	Alice 가 자기가 쓴 게시글 수정	Alice 가 자기가 쓴 게시글 수정	200 / 해당 없음	**PASS**	
T06	Alice 가 자기가 쓴 게시글 삭제	Alice 가 자기가 쓴 게시글 삭제	204 / 해당 없음	**PASS**	

HTTP 응답 확인 기록

예상 코드와 실제 코드를 대조하고, 201·204·401 의 헤더와 본문도 확인하세요.

숫자는 Network 나 API 클라이언트에서 직접 관찰한 값을 기록합니다. 본문에 있는 숫자만으로 판단하지 않습니다. 한 항목이 여러 요청을 포함하면 행을 추가하세요.

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
정상 조회·로그인·수정	**200**	[기록]	[기록]
새 글 등록	**201 및 Location**	[기록]	[기록]
본인 글 삭제	**204 및 빈 본문**	[기록]	[기록]
입력 오류·추가 작성자 필드	**400 BAD_INPUT**	[기록]	[기록]
잘못된 계정	**401 INVALID_CREDENTIALS**	[기록]	[기록]
토큰 없음·헤더 형식 오류	**401 TOKEN_REQUIRED**	[기록]	[기록]
변조·클레임 오류	**401 INVALID_TOKEN**	[기록]	[기록]
정상 서명의 만료 토큰	**401 TOKEN_EXPIRED**	[기록]	[기록]
다른 작성자의 변경	**403 FORBIDDEN**	[기록]	[기록]
없는 글	**404 NOT_FOUND**	[기록]	[기록]

[보호 API 401 의 WWW-Authenticate: Bearer 확인 결과와 증거]

[401 과 403 에서 화면이 각각 어떻게 달라지는지 설명]

[실패한 수정·삭제 뒤 공개 조회로 데이터 불변을 확인한 결과]

[토큰 없이 없는 글 수정 401 / 정상 토큰으로 없는 글 수정 404 의 실제 결과]

테스트 증거

캡처에는 테스트 ID 와 실제 응답을 표시하세요. 토큰 원문과 서버 비밀 키는 가립니다.

로그인과 토큰 전송

[증거 1: 로그인 성공과 토큰이 첨부된 요청. 토큰 원문은 가리세요.]

[화면 캡처 또는 요청·응답 삽입]

다른 작성자의 수정과 삭제

[증거 2: Bob 이 Alice 의 글 수정·삭제를 시도한 서버 응답]

[화면 캡처 또는 요청·응답 삽입]

토큰 변조와 만료

[증거 3: 토큰 변조·만료 시 서버 응답과 화면 처리]

[화면 캡처 또는 요청·응답 삽입]

자동 검사와 로컬 배포 증거

[자동 검사 결과를 첨부했다면 실행 명령과 의미를 설명]

[빌드 성공 로그 / 4173 번 API 연결 / 재시작 유지의 화면 또는 파일 경로]

AI 활용과 참고 자료

사용 범위, 실제 반영 위치와 직접 확인한 내용을 구분해 기록하세요.

AI 사용 여부: [사용 / 미사용]

사용하지 않았다면 미사용이라고 명시하고 참고한 수업 자료·공식 문서의 출처를 작성하세요. 사용했다면 전체 사용 범위를 요약하고 대표 사례 2~3 건을 작성하세요. 사용이 1 건이면 그 1 건만 적습니다. 대화 전체 제출은 필요하지 않습니다.

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식
제미나이	코드 문법 수정 내용 및 보고서 내용 설계	웹서비스설계 강의 자료 1~7 주차 / 과제 리포트 / 실습 영상	수정 후 반영

대표 사례

도구 / 날짜: 제미나이 / 2026.10.28

요청한 내용: Spring Boot 백엔드에서 발생할 수 있는 비즈니스 예외(인증 실패, 권한 없음, 유효성 검증 실패 등)를 일관된 JSON 에러 구조(code, message)와 표준 HTTP 상태 코드로 반환하는 전역 예외 처리 구현 요청

AI 가 제안한 내용: @RestControllerAdvice 및 @ExceptionHandler 를 활용한 GlobalExceptionHandler 클래스 구현 코드 및 CustomException 구조 제안

실제 반영 위치: backend-spring/src/main/java/.../exception/GlobalExceptionHandler.java

내가 확인하고 수정한 내용: AI 가 초기에 제시한 에러 응답 구조에 과제 요구사항에 명시된 필수 에러 코드(INVALID_CREDENTIALS, TOKEN_REQUIRED, TOKEN_EXPIRED, FORBIDDEN, BAD_INPUT, POST_NOT_FOUND)가 일부 누락되어 있음을 확인하고, 과제 스펙과 1:1 로 일치하도록 Enum 기반의 ErrorCode 클래스를 정의하여 예외 타입을 수동으로 직접 보완함.검증 방법과 결과: [테스트 ID / 요청 / 실제 응답 / 수정 전후 비교]

[필요한 만큼 사례 블록 복제]

참고 자료

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
[수업 자료 포함]	[파일/함수]	[작성]

이해도 확인과 회고

각 질문에 본인 코드의 동작을 근거로 답하세요.

1. 요청 본문의 authorId 를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

클라이언트가 전송하는 요청 본문(Request Body)이나 파라미터의 authorId 는 사용자가 Postman, F12 개발자 도구, cURL 등을 통해 얼마든지 위변조하여 전송할 수 있습니다. 만약 서버가 요청 본문의 authorId 를 그대로 신뢰하여 권한을 판별한다면, 공격자가 로그인 후 본문의 authorId 를 타인의 ID 로 주작하여 타인의 글을 무단으로 수정·삭제하거나 작성자를 위조하는 심각한 보안 취약점(BOLA / IDOR)이 발생합니다.

wtAuthenticationFilter 에서 서버 측 비밀키로 서명이 검증된 JWT 의 클레임(Claim)에서 직접 추출한 userId 를 SecurityContextHolder 에 저장합니다. 이후 PostService 의 updatePost 및 deletePost 메서드에서 DB 에 저장된 실제 게시글의 post.getAuthorId()와 서버가 직접 검증한 인증 객체의 currentUserId 를 $1:1$로 비교하여 다를 경우 즉시 403 FORBIDDEN 예외를 발생시켜 차단합니다.

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

프론트엔드에서 수정/삭제 버튼을 숨기는 것은 단지 UI/UX 차원의 단순한 1 차적 안내일 뿐입니다. 공격자는 브라우저의 F12 개발자 도구를 통해 버튼의 display: none 을 해제하거나, 버튼 클릭과 상관없이 API 엔드포인트(PUT /api/posts/{id})로 직접 HTTP 요청을 쏠 수 있습니다. 따라서 프론트엔드 제어는 보안 구실을 할 수 없으며, 반드시 서버 측에서 최종 권한 검증을 수행해야 합니다.

backend-spring 내 PostService.java 파일의 **updatePost()** 및 **deletePost()** 함수에서 해당 검사를 수행합니다.

3. JWT decode 와 verify 의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

Decode: Base64 로 인코딩된 JWT 의 Header 와 Payload 를 단순히 사람이 읽을 수 있는 JSON 형태로 복호화하는 과정입니다. 비밀키(Secret Key)가 없어도 누구나 수행할 수 있으며, 토큰이 위변조되었는지 여부는 알 수 없습니다.

Verify: 서버가 가지고 있는 고유한 비밀키(Secret Key)를 사용하여 Header 와 Payload 를 조합해 서명(Signature)을 다시 계산해 보고, 토큰에 첨부된 서명과 일치하는지 검증하는 과정입니다. 이 과정을 통해 위변조 여부 및 만료 시간(exp)을 판단합니다.

내 코드는 JwtProvider.java 에서 JJWT 라이브러리의 Jwts.parserBuilder().setSigningKey(secretKey).build().parseClaimsJws(token)를 호출합니다. 단순 Decode 가 아닌 **Verify** 를 수행하여, ① HS256 알고리즘 서명 위변조 여부, ② 토큰 유효기간 만료 여부(exp), ③ 필수 클레임(userId) 존재 여부를 모두 엄격히 검증합니다.

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

로그아웃: 프론트엔드 자바스크립트 메모리에 보관 중이던 authToken 변수를 null 로 초기화합니다.

토큰 만료: 서버 측 JwtProvider 에서 정의한 유효 시간(1 시간 / 테스트 시 3 초)이 경과하여, 클라이언트가 해당 토큰으로 요청을 보내도 서버에서 401 TOKEN_EXPIRED 응답을 반환하게 됩니다.

새로고침: 브라우저의 자바스크립트 실행 환경이 재시작되면서 메모리 내 authToken 변수가 초기화되어 비로그인 상태로 전환됩니다.

이미 복사된 토큰: 무상태(Stateless) JWT 특성상, 클라이언트에서 로그아웃을 하거나 새로고침을 하더라도 토큰 자체가 유효기간 내에 있고 서명이 올바르다면 이미 복사된 토큰은 유효합니다. 이 토큰으로 API 요청을 보내면 서버는 여전히 정상 응답(200 OK)을 줍니다. (이를 방지하기 위해 짧은 만료 시간 설정 및 Blacklist/Refresh Token 도입이 필요합니다.)

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

오류 및 증상: T06 / T10 시나리오 테스트 수행 중, 빈 입력값(빈 이메일/비밀번호 또는 빈 제목/내용)을 넣었을 때 백엔드로부터 400 BAD_INPUT 응답이 와야 하는데, 프론트엔드 화면에서 제출 자체가 클릭되지 않거나 반응이 없는 현상이 발생함.

원인 확인: 프론트엔드 HTML Template 내 <input required /> 속성이 지정되어 있거나, JS 호출 함수 시작 지점에서 if (!title) return; 과 같이 클라이언트 1 차 유효성 검사가 작동하여 백엔드로 API HTTP 요청 자체를 보내지 않고 브라우저단에서 1 차 차단하고 있었음을 F12 Network 탭 요청 기록 부재로 확인함.

현재 남은 문제와 다음에 개선할 점: [없으면 없음으로 표시]

제출 확인: [v] 실제 코드와 설명 일치 [v] 테스트 직접 수행 [v] AI 활용 및 출처 기록 [v] 비밀 키와 토큰 제거

GitHub 확인: [v] 루트 README 작성 [v] 보고서 PDF 와 증거 업로드 [v] README 문서 링크 확인 [v] 교수자 접근 가능 [v] 저장소 URL 과 커밋 SHA 제출